Contents

1 Organizing your projects 1
1.1 Two templates 1
1.2 A Note on Paths 2

1 Organizing your projects

- Reading: @NobleQuickGuideOrganizing2009
- See also: https://support.rstudio.com/hc/en-us/articles/200526207-Using-Projects
- Good project organization will
  - Keep your project self-contained, making it easy to find files as needed
  - Play nicely with version control
  - Self-document key relationships between project files

While we're at it, put your work in folders, not scattered across your desktop.

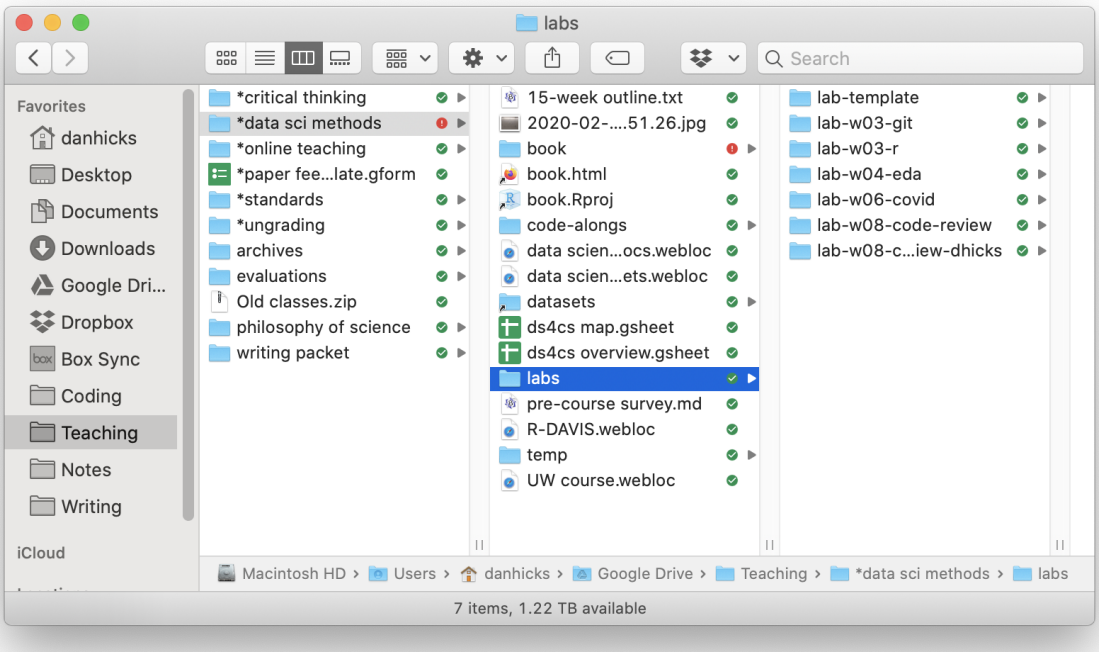

Figure 1: Folder organization is your friend. Most of my work lives in Google Drive for automatic backups. I have top-level folders for teaching, coding, and writing projects. Then each project or collection of projects lives in its own folder. I take advantage of OS features such as shortcuts, highlighting, and special characters in file names to make it easy to find frequently-used things. An 'Archives' folder is good for tucking old projects out of the way.

1.1 Two templates

1.1.1 @NobleQuickGuideOrganizing2009

- Includes code for

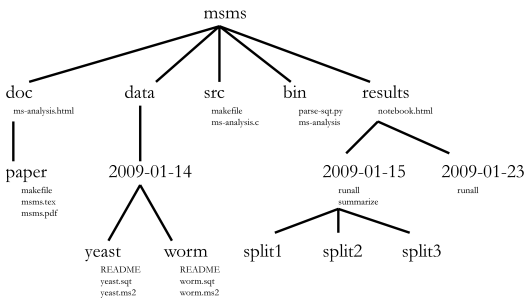

Figure 2: Noble's [-@NobleQuickGuideOrganizing2009] sample folder structure is designed for experimental biologists.

  - running experiments
  - cleaning and analyzing data
  - typesetting a LateX file into a PDF
- Other features
  - notebook file, updated regularly
  - chronological folders for experimental runs
  - folders for compiled code and their corresponding source files

1.1.2 DJH's Project Template

- https://github.com/dhicks/project_template
- Configured as a GitHub "template," making it easy to create new repositories for new projects
- Designated folders for data, plots/outputs, and utility functions
- File naming convention
  - Automatically orders analysis scripts
  - Coordinates plots, outputs, and intermediate data with the script that creates them
- R package infrastructure
  - Enables one-liner to load the utility functions in `R`: `devtools::load_all('..')`

1.2 A Note on Paths

- Windows and Unix-based systems write paths differently
- Use `file.path()` or the `here` package to construct paths
- `..` in a path means "go up to the parent folder"
  - so `../data/00_raw_data.csv` goes up one level (eg, from the `scripts` folder), then down to the `data` folder, then the file `00_raw_data.csv`